

Tecnun
Universidad
de Navarra

MACHINE LEARNING ASSIGNMENT

German Credit Risk Classification

Aitor Marcalain Mauleón
Machine Learning
Tecnun 2025-26

INDEX

1. Context **3**
2. Objective **3**
3. Dataset **3**
4. Tasks **4**
Task 1: Exploratory Data Analysis **4**
Data Overview 4
Exploratory Data Analysis (EDA) 4
Task 2: Data preprocessing 7
5. Model selection **8**
6. Linear Classification Models **8**
Tuning the regularization parameter C 9
Logistic regression 9
Linear Support Vector Machine (Linear SVM) 10
7. Non-Parametric Model: k-Nearest Neighbors 12
8. Comparative Analysis and Discussion **13**
9. Conclusion and final recommendation **14**

1. Context

For this assignment, I've been tasked with predicting whether a bank applicant is a good or bad credit risk. To solve this, I need to build and compare various classification models.

2. Objective

The primary objective of this project is to build a binary classifier that successfully predicts whether an applicant is a bad credit risk , using "class" as the target variable (good=0, bad=1). To achieve this, I will perform an exploratory data analysis (EDA) , apply proper data preprocessing using Pipelines, and build and tune various classification models. Finally, I will compare the performance of these models to justify a final recommendation for the most effective risk prediction solution

3. Dataset

To train and evaluate these models, I am using the open German Credit dataset, also referred to as Statlog German Credit.The dataset is publicly available via OpenML (under the name credit-g, dataset ID 31) and the UCI Machine Learning Repository. It contains exactly 1,000 applicant records. These records are described using a mixture of numerical and categorical attributes, resulting in approximately 20 features depending on the final representation used.

In order to do this, after importing into my colab all the necessary tools, I fetched this data directly into my Google Colab environment using the *fetch_openml* function. I stored this in the variable *Xy* . After, I separated this variable into *y_raw* (containing the “class” target variable) and X (the features of each sample). Since *y_raw* is not a numeric variable, we need to convert this to numerical mapping (good=0, bad=1) to the target column. Finally with *y.unique()* we ensure that the only values that y takes are 0 and 1.

4. Tasks

Task 1: Exploratory Data Analysis

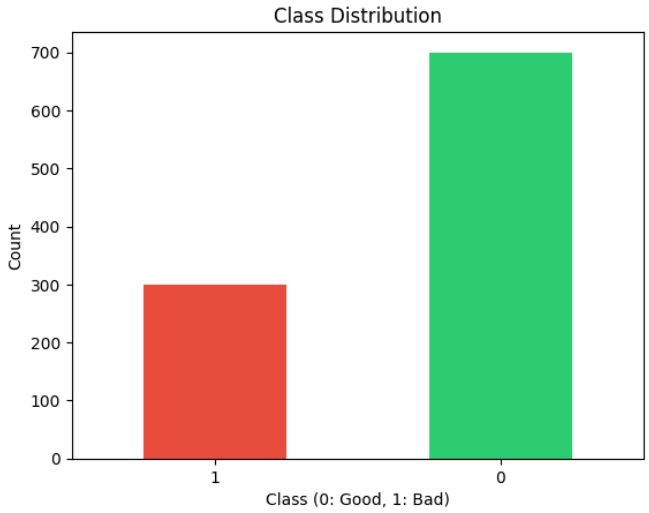
Data Overview

Using *X.shape* we can see that the samples consist on 1000 data points with 20 features each. By adding *X.describe* we see the distribution of those features. Furthermore, *X.isna().sum().sum()* tells us that there are no NaN values on the dataset. Same with y.

Exploratory Data Analysis (EDA)

We now want to see how the features are distributed, so we can see whether at first sight, there is a noticeable difference between the good and bad applicants. Also, some features may need some preprocessing.

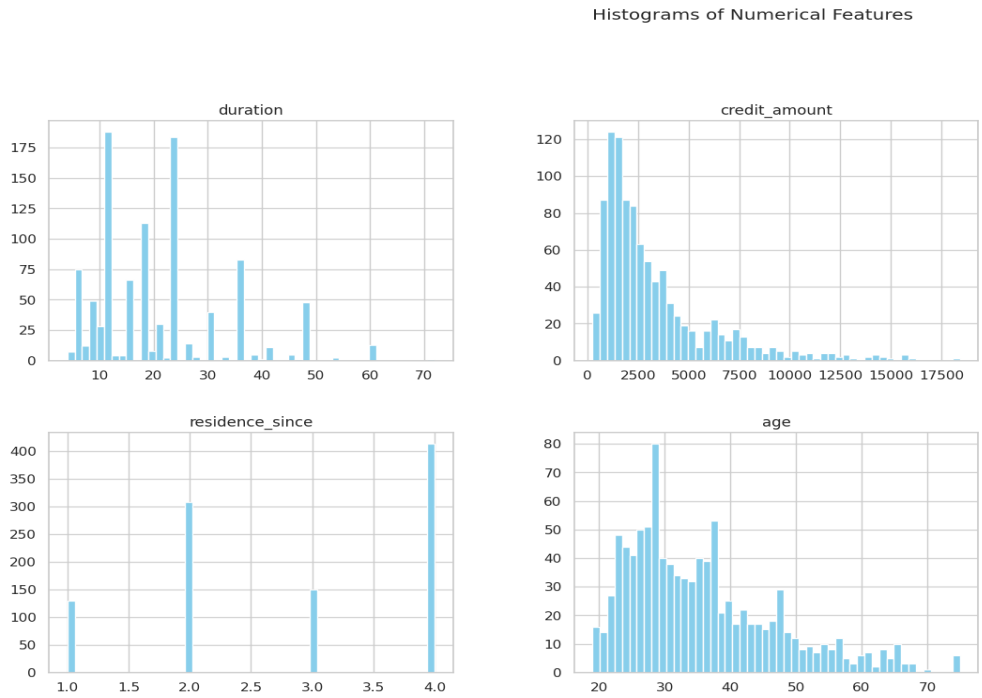
After counting how many applications were good or bad, we get the following:



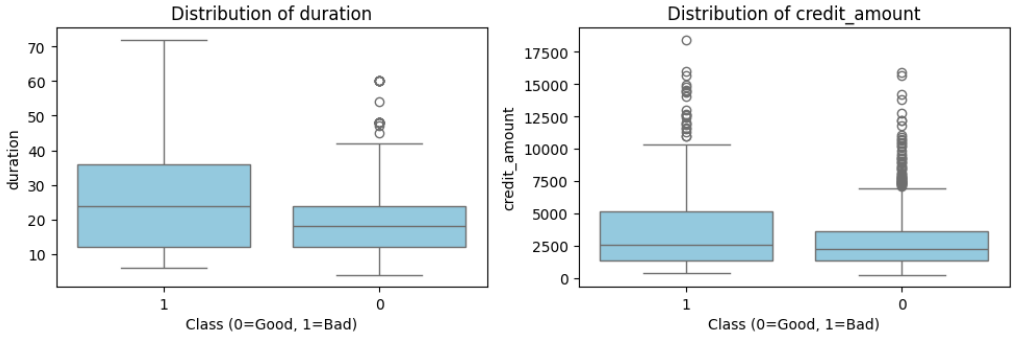
As we can see, the majority of the applicants returned the credit. However, there are a lot of bad applicants who requested credit. This is costly and in the future the bank should reduce this, and that is the purpose of this project, in fact. We also need to consider that this means that the data is unbalanced, therefore if the classification guesses all good it would still get a 70% right.

Now, we plot a histogram of the numerical features so we can analyze their distribution. From that graphic, we conclude that a scaling of the features will

be needed further in the project, taking into account that scales are unbalanced, so features with higher values would be given more importance. These are some of the histograms of numerical values where we can appreciate the scale variance. For example, the *credit_amount* variable will be given too much importance

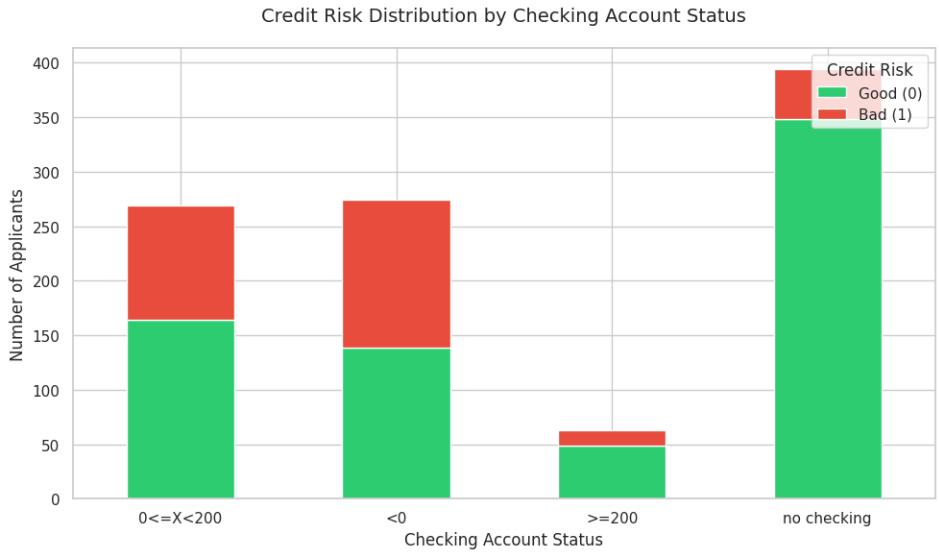


To further investigate the relationship between numeric features and credit risk, I generated boxplots for each quantitative variable, separated by the target class (0 for good, 1 for bad). This step is crucial for identifying which features exhibit significant differences in their medians or distributions across the two groups.



Both features represented above have a significant difference in their medians. This is logical, since we expect that the higher the duration and credit amount of the loan the higher is the probability to obtain a bad applicant.

Finally, I followed the same approach for a categorical variable:



Regarding the data given, we see that for example, the majority of the clients who were bad applicants had a negative checking account. We could have known by intuition as well. In order to apply our models to the categorical variables we should use one hot encoding, which I will elaborate more further on.

Task 2: Data preprocessing

Before applying any transformations, I divided the dataset into training and testing sets to evaluate the model ability to generalize to new, unseen data and prevent overfitting. Because if we test the model on the same data we used to train the model it may overfit. To achieve this, I used the *train_test_split* function with the following key parameters:

- *test_size*=0.20: Reserves 20% of the data for the final evaluation while keeping 80% for model training.

- *random_state*=42: Sets a random seed to ensure the data split is the same across different runs.
- *stratify*=y: This is really important for our imbalanced dataset, ensuring that the original proportion of "good" and "bad" credit risks is maintained.

Printing the proportion of good/bad in the training and test sets we get:

```
X_train shape: (800, 20)
X_test shape: (200, 20)
Proportion of Bad credit in Train: 0.30
Proportion of Bad credit in Test: 0.30
```

Without *stratify=y*, we leave the data splitting entirely up to chance. Since our dataset is imbalanced (about 70% "good" credit and 30% "bad" credit), a random split might accidentally group almost all the "bad" cases into the training set and leave very few for the test set. If our test set ends up with mostly "good" cases, our model could score a very high accuracy just by guessing "good" every single time, which would completely hide its inability to detect real credit risks. Using stratify=y solves this problem by guaranteeing that both the training and testing sets keep the exact same 70/30 ratio as the original data, ensuring the model is trained and evaluated fairly.

Defining a pipeline is the next step. The first thing is separating the numerical and categorical data, so we can apply different processing to each of them. For the numerical values we standardize the data (reason explained before). We use the following formula:

$$z = \frac{x - \mu}{\sigma}$$

The categorical pipeline uses one hot encoding, and *handle_unknown*='ignore' ensures that if a rare category appears in the test set that wasn't in the training set, it won't crash the code. This encoding creates a new binary column for each unique category within a feature, assigning a 1 if that category is present and a 0 otherwise

With the *ColumnTransformer*, I am simply putting our preprocessing instructions together into a single object. I am mapping the numerical columns to their specific numeric transformer and the categorical columns to their categorical transformer. This step doesn't actually calculate the data yet; it just

creates a set of rules that we will use later on (which I will explain in another section).

5. Model selection

- What assumptions does the Perceptron make? That the data is bounded and linearly separable by a hyperplane
- When is convergence guaranteed? Convergence is guaranteed if and only if the dataset is perfectly linearly separable (positive margin) and the data is bounded.
- Is it likely appropriate for this dataset? Why or why not? The perceptron is going to be highly unlikely to be appropriate for this data set. Although the data is bounded, two identical inputs can have different outputs. Furthermore, real-world data is almost never linearly separable, especially in this case, knowing that human behavior is complex

6. Linear Classification Models

I used a pipeline to link all my data transformations—both for numerical and categorical features—directly to the Logistic Regression model. The most important reason for doing this is to prevent a serious mistake called "data leakage," which directly leads to overfitting.

If I had scaled the numerical features before doing the train/test split, the transformations would have accidentally learned information from the test set. This means the model would "cheat" during training by already knowing information about the test data, resulting in a fake high score. By using a Pipeline, I guarantee that the preprocessor only learns its rules strictly on the training data. Later, it just applies those exact same mathematical rules to transform the test set (the mean and std in this case), preventing the model from overfitting to leaked information.

Tuning the regularization parameter C

In this section I used GridSearchCV to automatically test different values for C (the parameter that controls how complex the Logistic Regression model is allowed to be). Instead of just guessing, this process finds the best point, where the model learns the patterns without just memorizing the training data (If C is too low, the model might oversimplify; if it's too high, it might memorize the training data and overfit).

In order to find this value, first I provided the c values that we will test. With *StratifiedKFold* I trained the model on 4 pieces and tested it on the 1 remaining piece. It repeats this 5 times, so every piece gets a turn being the test set. The final score is the average of all 5. With this function I also guarantee that the 70/30 class balance is strictly maintained across every test during the search.

The *GridSearchCV* searches for the best score. It is important to note that I chose the F1 score instead of standard accuracy because, as I mentioned, the dataset is imbalanced, which makes accuracy a highly misleading metric. To prevent this, the F1 score evaluates the model based on the harmonic mean, a balance of precision (of all predicted bad applicants, how many are truly bad?) and recall (of all actual bad applicants, how many did we detect?). After proceeding with the search, these were the results.

```
Best C value found: 10
Best cross-validation F1 score: 0.5035
```

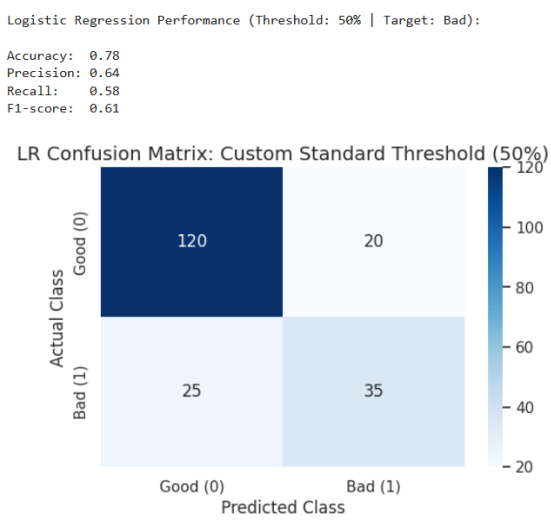
Logistic regression

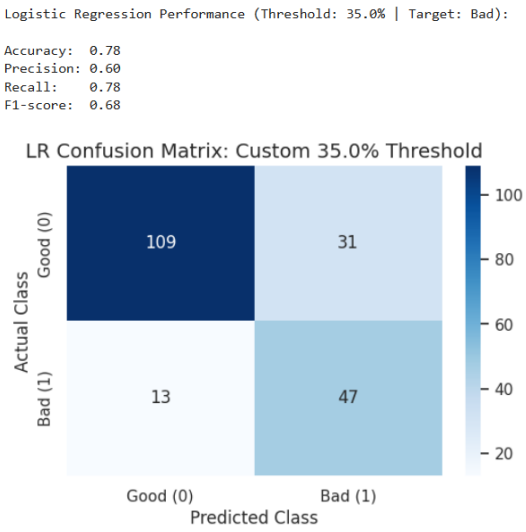
For this project, the first model used is Logistic Regression, which is a classification algorithm that calculates the probability of an applicant being bad (not returning the loan), rather than just outputting a “yes” or “no”. By default, the model uses a 50% probability threshold to classify an applicant as a risk.

However, when I evaluated my model using this standard threshold, it achieved a recall score of only 0.58. In the context of credit risk, this is a highly undesirable result because it means the model missed 42% of the actual bad applicants, who would have been approved for loans. Since giving credit to a bad applicant (a false negative) is the most expensive mistake a bank can make, I needed to make the model more sensitive to risk.

To solve this, I manually lowered the model's decision threshold. By doing this, the outputs that have a higher than 35% probability of being bad are also classified as bad. I chose this value with cross-validation on the training set: using the out-of-fold predictions, I kept the threshold with the highest recall whose accuracy was at most 3 points below the one obtained with the default 50% threshold. This way, the test set is only used for the final evaluation and there is no data leakage. So even though this may classify some good applicants wrongly, we reduce the number of false negatives, which are far more costly for the bank. Consequently, after doing that change, we obtained a 0.78 recall, while maintaining the exact same accuracy as before. The only thing that worsened

was the precision, but as I said, this is less costly. We can also see this improvement in the confusion matrix, where we see that the false negatives (bad applicants predicted as good) went down from 25 to just 13.





Linear Support Vector Machine (Linear SVM)

For the next model of the project, I implemented a Linear Support Vector Machine (LinearSVM). Unlike Logistic Regression, which calculates the statistical probability of an applicant being bad, SVM takes a purely geometric approach. It maps all the applicants as data points in a space and attempts to draw a straight line (known as a hyperplane, boundary) that separates the good applicants from the bad ones with the widest possible margin. Because it works with the distance to this boundary rather than probability calculations, SVM can sometimes be more effective at defining strict and complex borders between classes. However, this also means it outputs rigid classifications rather than risk percentages.

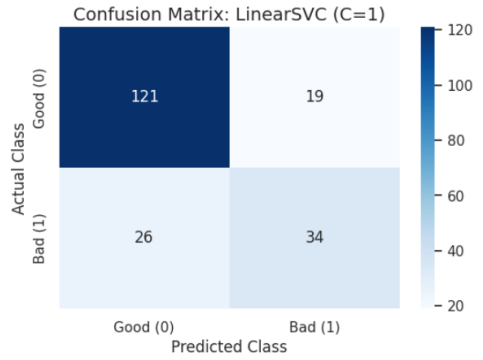
Since this model is completely different to Linear Regression we need to find the new best C parameter. To achieve this, we do exactly the same search, *GridSearchCV* that we did for LR, but using our new model. We obtain the following:

```
Best value of C found for SVM: 1
Best F1 score in cross-validation: 0.5036
```

As we can see, the best c value found is different from the previous one. We then proceed to test the model using that value. The results are practically identical to the ones we got using linear regression. Therefore, the recall is still dangerously low. Regarding this model, to get a higher recall, we lower the threshold so we are physically sliding that dividing line deeper into the "good" area. As with Logistic Regression, the new threshold (-0.25) was selected with cross-validation on the training set using the same rule. We do this because we want the model to be more sensitive, purposefully capturing those borderline cases that the strict default line would have otherwise ignored (same as before but using another approach). By shifting the separator, we expand the actual area that the model classifies as "bad." Data points that previously landed just inches on the "good" side of the default line are now inside the new boundary. Consequently, the model successfully classifies more true "bad" cases, giving us a higher recall:

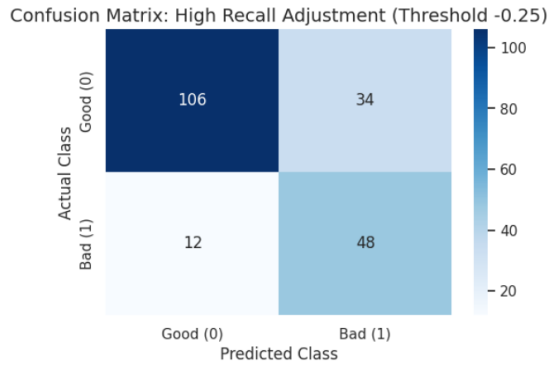
```
Linear SVC in test (C=1 | Target: Bad):

Accuracy:  0.78
Precision: 0.64
Recall:    0.57
F1-score:  0.60
```




```
LinearSVC Performance with Adjusted Threshold (-0.25):

Accuracy:  0.77
Precision: 0.59
Recall:    0.80
F1-score:  0.68
```



7. Non-Parametric Model: k-Nearest Neighbors

The K-Nearest Neighbors (KNN) algorithm is one of the simplest and most intuitive machine learning models. Instead of trying to calculate a complex mathematical boundary to separate classes, it simply "memorizes" the entire training dataset. When you ask it to predict the class of a new, unseen data point, the model looks at the “k” closest training points (the neighbors) surrounding it. The new point is then assigned to whichever class has the majority vote among those neighbors. The model makes zero

assumptions about the underlying shape or distribution of the data. Unlike Logistic Regression or SVM, which assume they can draw a geometric line.

When tuning a KNN model, these are the main settings that dictate how it makes its decisions:

- *n_neighbors* (k): This is the exact number of nearby data points the algorithm will check before making a prediction. I chose this by using *GridSearchCv* once again, searching from odd numbers so we can avoid ties.

- *weights*: This determines how the neighbors' votes are counted.

    - uniform (default): Every neighbor gets exactly one equal vote, regardless of how close or far away it is from the new point.

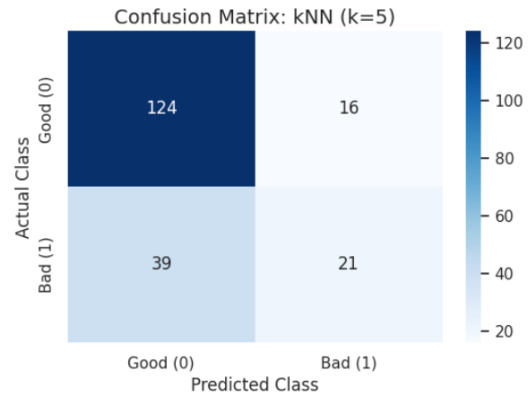
- *p*: This defines the actual mathematical formula used to measure the physical distance between data points in your feature space.

    - p=2 (Euclidean distance): The default value. It calculates the straight-line distance between two points.

After using this model to classify the test points we get this scores and confusion matrix:

```
kNN Performance (k=5 | Default 50% Threshold | Target: Bad):

Accuracy:  0.72
Precision: 0.57
Recall:    0.35
F1-score:  0.43
```



As the confusion matrix shows, this mo rious ones. It only achieved a recall of 0.35, a l bad loans (false negatives). The main reas nced dataset. Since 70% of our applicants are good, a risky applicant is often surrounded by 'good' applicants just by chance. The 'good' majority causes the model to just guess 'good' most of the time. While LR and SVM use math to draw a clear line separating the groups, kNN just gets overwhelmed by the higher number of good loans.

8. Comparative Analysis and Discussion

- Why can Logistic Regression and Linear SVM give different results even though both are linear? (answer given for the standard cases, 50% threshold) While they both end up with a line which separates good from bad loans, since they use entirely different rules to get there the lines give different results. Logistic Regression will consider each and every applicant to find a probability, so a large group of "safe" loans far away can actually push and adjust the line. Instead SVM exclusively looks at the borderline cases (in the support vectors) and separates those with a margin as wide as possible.

- Which models provide probability estimates? Why might this matter in credit risk? Logistic regression does this. Because in credit risk, for an applicant, it is not enough to only know a prediction of 0/1. Logistic regression gives us a probability estimate of that applicant being bad. Therefore, we can make different decisions on whether to give them the loan depending on that probability. For example, if the prediction for an applicant was 0.49, SVM might have classified that client as good and we would have accepted the loan. However, since we want to avoid giving loans to bad applicants, analyzing the probability we would have concluded not to give the loan since it was too risky.

- What happens when C is very large or very small? . When C is set very small, the model is heavily restricted; it draws a very simple decision boundary, but risks underfitting by ignoring important complex patterns. On the other hand, when it is very large, the model is allowed to be highly complex. It will try to perfectly classify every single training example, which leads to overfitting—meaning it memorizes the training data but performs poorly on new data. By using cross-validation (with *GridSearchCV)* to test multiple values of C, and searching for the best F1 score, I was able to find the perfect tradeoff between under and overfitting.

- How does increasing k affect bias and variance in kNN? Similarly to what happens with C, when k is very small the model has high variance and low bias. It only listens to the 1 or 3 closest people. If one of those neighbors happens to be a weird outlier, the model blindly follows them. It draws a complex boundary line that memorizes every point in the training data. This leads to overfitting. When k is very large, the model has high bias and low variance. It listens to so many people that the patterns near the point we are predicting get completely overlapped by the majority vote. It draws a simple line (or in our case just predicts "good" for everyone because "good" is the majority). This leads to underfitting.

9. Conclusion and final recommendation

- Which model would you recommend and why?: After having analysed every model and their advantages, I concluded that Logistic Regression would be the most appropriate classificator for this task. As we saw, kNN was not great for our project because the dataset is imbalanced. Then, as I have already mentioned, between SVM and LR, the second one provides probability estimations which are really convenient for this task. In fact, we can make decisions after seeing those probabilities, rather than just in the rigid 0/1 estimates that SVM provides us. Moreover, after having improved our logistic regression model, we obtain a recall of 0.78. This avoids the risk of approving loans to bad applicants, while maintaining an accuracy of almost 80%.

  Therefore, when we get a new applicant we will apply our model, and if the probability of him/her being bad is lower than 35% we would approve the loan. Finally, this model also allows us to explain why one output could not have been what eventually happened. Imagine that we give a client a loan but ends up being a bad customer. Then, looking at our results we could see that the probability of that client being bad was only 10% and explain that it was reasonable to approve that loan in the first place.